

AWiR – ćwiczenie 9

Temat:

Spring Integration – serwer HTTP z inbound **gateway**.

Cel ćwiczenia:

Rozszerz aplikację z **Ćwiczeń 4–8** o serwer HTTP zrealizowany w **Spring Integration**, który:

- wystawia **endpoint HTTP** (np. POST /si/users-sync),
- odbiera JSON z danymi User (tak jak REST /api/users),
- przekazuje żądania na **kanał Spring Integration**,
- wywołuje istniejący UserService w .handle(...),
- **zwraca w odpowiedzi HTTP JSON zapisanej encji User** (tak jak klasyczne REST API).

To ćwiczenie ma pokazać różnicę między klasycznym REST (@RestController) a podejściem *message-driven* z użyciem **HTTP inbound gateway** w Spring Integration.

Niezbędne narzędzia:

1. DE: STS 4 / IntelliJ / VS Code
2. Maven 3.9+
3. JDK 21
4. Spring Boot 3.3.x
5. Znajomość projektu z ćw. 4–8 (UserService, REST, Security, Mail, SOAP, itd.)

Dotychczasowy projekt zawierał:

Masz już projekt:

- User, UserRepository, UserService,
- REST API: UserRestController pod /api/users/**,
- MVC: UserController pod /users/**,
- Security:
 - /api/** – HTTP Basic, stateless,
 - MVC – formLogin + remember-me + Google OAuth2,
- SOAP i mail.

W tym ćwiczeniu dodaj:

- **nowy endpoint HTTP**, ale zrobiony **nie przez @RestController**, tylko przez **Spring Integration**.

Zmiany w strukturze projektu

src/main/java/edu/zut/awir/

├── AwirApplication.java

├── config/

│ ├── SecurityConfig.java

│ ├── WebServiceConfig.java

│ ├── AsyncConfig.java

│ └── **IntegrationConfig.java**

NOWE @EnableIntegration (puste)

```

├── model/
│   └── User.java
├── repository/
│   └── UserRepository.java
├── service/
│   ├── UserService.java
│   └── MailService.java
├── integration/
│   └── UserIntegrationFlow.java    # NOWE: flow z HTTP inbound gateway

```

```

src/main/resources/
├── application.properties
└── templates/...

```

1. Rozszerzenie zależności w pliku pom.xml

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-integration</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.integration</groupId>
  <artifactId>spring-integration-http</artifactId>
</dependency>

```

2. Włączenie Spring Integration

src/main/java/edu/zut/awir/config/IntegrationConfig.java

```

package edu.zut.awir.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.integration.config.EnableIntegration;

@Configuration
@EnableIntegration
public class IntegrationConfig {
}

```

3. HTTP inbound gateway → UserService

src/main/java/edu/zut/awir/integration/UserIntegrationFlow.java

```

package edu.zut.awir.integration;

import edu.zut.awir.model.User;
import edu.zut.awir.service.UserService;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.integration.dsl.IntegrationFlow;
import org.springframework.integration.dsl.IntegrationFlows;
import org.springframework.integration.http.dsl.Http;

import java.util.Map;

@Configuration
@RequiredArgsConstructor
public class UserIntegrationFlow {

    private final UserService userService;

```

```
// Flow obsługujący żądanie HTTP i zwracający odpowiedź
@Bean
public IntegrationFlow userHttpGatewayFlow() {
    return IntegrationFlows
        .from(Http.inboundGateway("/si/users-sync")
            .requestMapping(m -> m
                .methods(HttpMethod.POST)
                .consumes("application/json"))
            )
        .requestPayloadType(User.class) // JSON -> User
        )
        .handle((User user, Map<String, Object> headers) -> {
            // zapisujemy użytkownika i ZWRACAMY zapisany obiekt jako odpowiedź HTTP
            return userService.save(user);
        })
        .get();
}
}
```

4. Security dla /si/users-sync

Rozbudowa SecurityConfig.

```
.authorizeHttpRequests(reg -> reg
    // ...
    .requestMatchers("/si/users-sync").hasAnyRole("USER", "ADMIN")
    // ...
)
```

5. Testowanie

- Uruchom aplikację.
- Wywołaj gateway (POSTMAN, CURL):

```
curl -u user:user123 -X POST http://localhost:8080/si/users-sync \
-H "Content-Type: application/json" \
-d '{"name":"Integration User","email":"integration@example.com"}'
```

6. Zadania do wykonania

- Dodaj zależności spring-boot-starter-integration i spring-integration-http do pom.xml.
- Włącz Spring Integration (@EnableIntegration).
- Zaimplementuj IntegrationFlow z Http.inboundGateway("/si/users-sync"), który:
 - przyjmuje JSON User w POST /si/users-sync,
 - wywołuje userService.save(user),
 - zwraca w odpowiedzi HTTP JSON zapisanej encji.
- Skonfiguruj Security tak, aby /si/users-sync było chronione (np. jak /api/users).
- Przetestuj wywołanie z użyciem curl / Postman i zweryfikuj wpis w bazie.